



Islamic University of Technology

Department of Computer Science and Engineering (CSE)

CSE 4205: Digital Logic Design

Summer, 2024–2025

Aashnan Rahman, Jr. Lecturer, CSE

Lecture 14

Registers and Counters

Contents

I	Registers	3
1	Introduction to Registers	3
1.1	Why D Flip-Flops Are Commonly Used	3
1.2	Clocking and Timing Requirements	4
2	Classification of Registers	4
3	Serial-In Serial-Out (SISO) Register	5
3.1	Clock-by-Clock Example	6
3.2	Left-Shift SISO Register	8
4	Serial-In Parallel-Out (SIPO) Register	9
5	Parallel-In Serial-Out (PISO) Register	10
5.1	Parallel-Load and Serial-Output Example	11
6	Parallel-In Parallel-Out (PIPO) Register	14
7	Bidirectional Shift Register	14
8	Universal Shift Register	16
8.1	Mode 00: No Change	17
8.2	Mode 01: Shift Right	18
8.3	Mode 10: Shift Left	19
8.4	Mode 11: Parallel Load	19
8.5	Complete Next-State Table	20
8.6	Combined Operation Example	20
9	Summary Comparison	21
10	How Many Clock Cycles Must We Wait?	21
10.1	Time to Load the Data	22
10.2	Time to Obtain the Output	22
II	Counters	23
11	Introduction to Counters	23
11.1	Why Counters Usually Use T Flip-Flops	23
11.2	Types of Counters	23
12	Three-Bit Asynchronous Up Counter	24
12.1	How the Negative-Edge Ripple Works	24
13	Three-Bit Asynchronous Down Counter	25
13.1	Way 2: Clock the Next Stage from $\overline{Q}$	26

14 MOD Counters	26
14.1 MOD-5 Counter and Its Reset Logic	27
14.2 Synchronous MOD Counters	28
15 Ring and Johnson Counters	28
15.1 Ring Counter	28
15.2 Johnson Counter	29
16 Counter Topics Already Covered in Class	30
17 A few things to note	30

Part I

Registers

1 Introduction to Registers

A **register** is a group of flip-flops used to store a binary word. Each flip-flop stores one bit, so an n -bit register contains n flip-flops and can hold one of 2^n different bit patterns. For example, four flip-flops can store one four-bit word such as 1011.

All flip-flops in a register normally share a common clock. Therefore, the stored word is updated synchronously at the active clock edge. Combinational logic may be placed before the flip-flop inputs to select operations such as hold, shift, and parallel load.

1.1 Why D Flip-Flops Are Commonly Used

For a D flip-flop,

$$Q_{n+1} = D.$$

Whatever value is present at D immediately before the active clock edge is stored at Q after the propagation delay. This direct relationship makes the data path easy to understand: connect each D input to the bit that should be stored next.

D before active edge	Q_{n+1}	Operation
0	0	Store 0
1	1	Store 1

Table 1: Characteristic table of a D flip-flop.

JK or SR flip-flops can also implement registers, but extra input equations are required. A T flip-flop is designed primarily to hold or toggle, so it is less convenient for loading an arbitrary data bit.

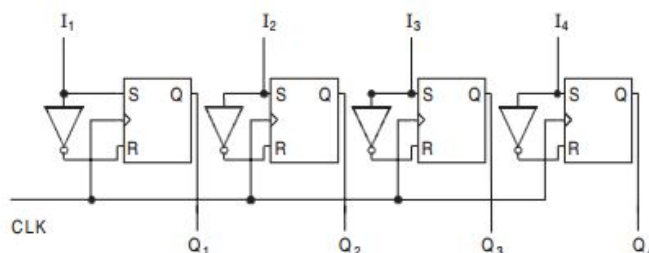


Figure 1: A four-bit register implemented with gated SR storage elements, as shown in the lecture slides. D flip-flops provide the same storage function with a simpler single data input.

1.2 Clocking and Timing Requirements

The data inputs must remain stable for the flip-flop's **setup time** before the active clock edge and for its **hold time** after the edge. All outputs then update after the clock-to-output propagation delay.

Term	Meaning
Setup time t_{su}	Minimum time the input must be stable before the active clock edge.
Hold time t_h	Minimum time the input must remain stable after the active clock edge.
Propagation delay t_{pd}	Time between the active clock edge and a valid change at the output.
Clear	Forces the stored word to zero, often independently of the clock.

Table 2: Important register timing and control terms.

Edge-triggered or master-slave flip-flops prevent a value from passing through several stages during one clock pulse. Every bit should move by only one position per active edge.

2 Classification of Registers

The letters in the standard names describe how data enters and leaves.

Type	Input	Output	Main purpose
SISO	Serial	Serial	Delay or transfer one bit per clock
SIPO	Serial	Parallel	Serial-to-parallel conversion
PISO	Parallel	Serial	Parallel-to-serial conversion
PIPO	Parallel	Parallel	Store or transfer a complete word

Table 3: Classification by input and output method.

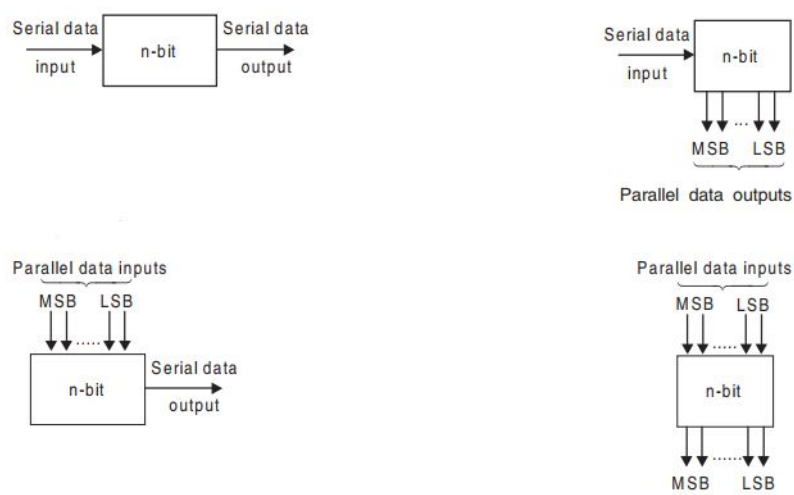


Figure 2: Block-level classification diagram from the lecture slides: SISO, SIPO, PISO, and PIPO.

Registers may also be classified by allowed direction:

- a **unidirectional shift register** shifts in only one direction;
- a **bidirectional shift register** can shift left or right;
- a **universal shift register** can hold, shift right, shift left, and load in parallel.

3 Serial-In Serial-Out (SISO) Register

In a SISO register, data enters through one serial input and leaves through one serial output. The source-note circuit below contains four positive-edge-triggered D flip-flops with a common clock.

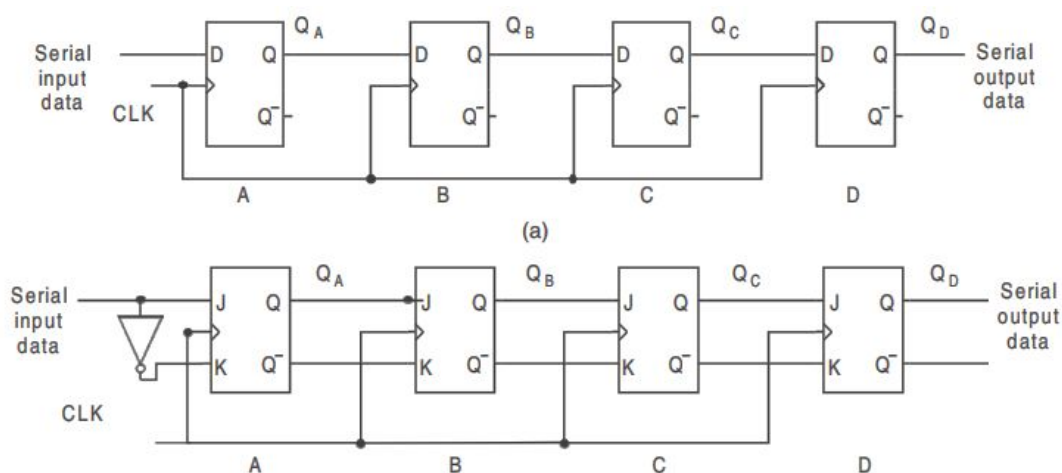


Figure 3: Lecture-slide comparison of a right-shift SISO register implemented with D flip-flops and with JK flip-flops.

The interconnections are

$$D_A = SI, \quad D_B = Q_A, \quad D_C = Q_B, \quad D_D = Q_C,$$

where SI is the serial input. Thus, at every active clock edge,

$$Q_A^+ = SI, \quad Q_B^+ = Q_A, \quad Q_C^+ = Q_B, \quad Q_D^+ = Q_C.$$

The arrows added in the handwritten page emphasize the path

$$SI \longrightarrow Q_A \longrightarrow Q_B \longrightarrow Q_C \longrightarrow Q_D \longrightarrow SO.$$

All transfers occur together at one clock edge; a bit does not pass through multiple stages during a single edge. In other words, at every active clock trigger, each stored bit is shifted by exactly one register position.

3.1 Clock-by-Clock Example

Suppose the register is initially $Q_A Q_B Q_C Q_D = 0000$. To obtain 1011 in the register, enter the bits LSB first: 1, 1, 0, 1.

In Table 4, color identifies a particular data bit rather than a register position: **first bit**, **second bit**, **third bit**, and **fourth bit**. Follow one color diagonally to see that the same bit moves one stage to the right on every clock edge. After edge 4, the complete word $Q_A Q_B Q_C Q_D = 1011$ is available in parallel. The additional clock edges show the same word leaving through the serial output; zeros are applied at SI during this output phase.

Active edge	Serial input SI	Q_A	Q_B	Q_C	Q_D	Serial output Q_D after edge
Initial	—	0	0	0	0	—
1	1	1	0	0	0	0
2	1	1	1	0	0	0
3	0	0	1	1	0	0
4	1	1	0	1	1	1
5	0	0	1	0	1	1
6	0	0	0	1	0	0
7	0	0	0	0	1	1

Table 4: Serial loading of 1011, followed by its serial output from the four-bit SISO register.

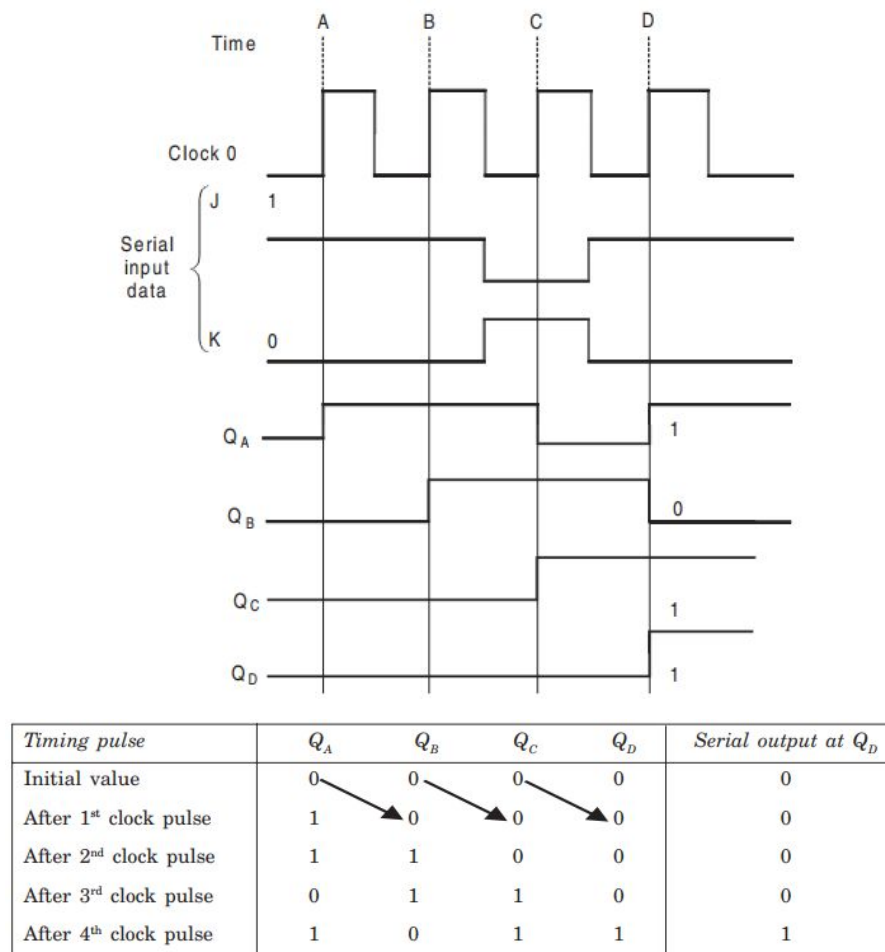


Figure 4: The lecture-slide waveform and its clock-by-clock table for loading 1011 into a right-shift register.

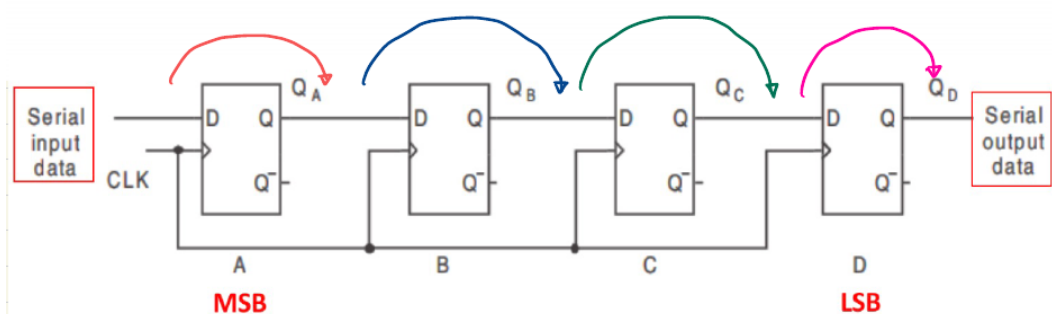


Figure 5: Annotated SISO data path. At every active clock trigger, each bit shifts by one position in the direction of the arrows.

3.2 Left-Shift SISO Register

Reversing the interstage connections reverses the shift direction. The serial input enters stage A at the right in the lecture-slide convention, and data moves $Q_A \rightarrow Q_B \rightarrow Q_C \rightarrow Q_D$ toward the serial output drawn on the left.

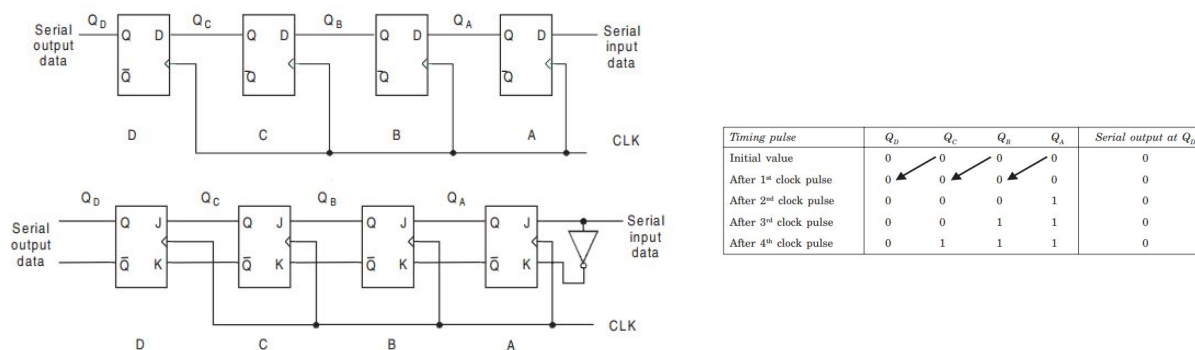


Figure 6: Left-shift SISO implementations and the corresponding data-movement table from the slides.

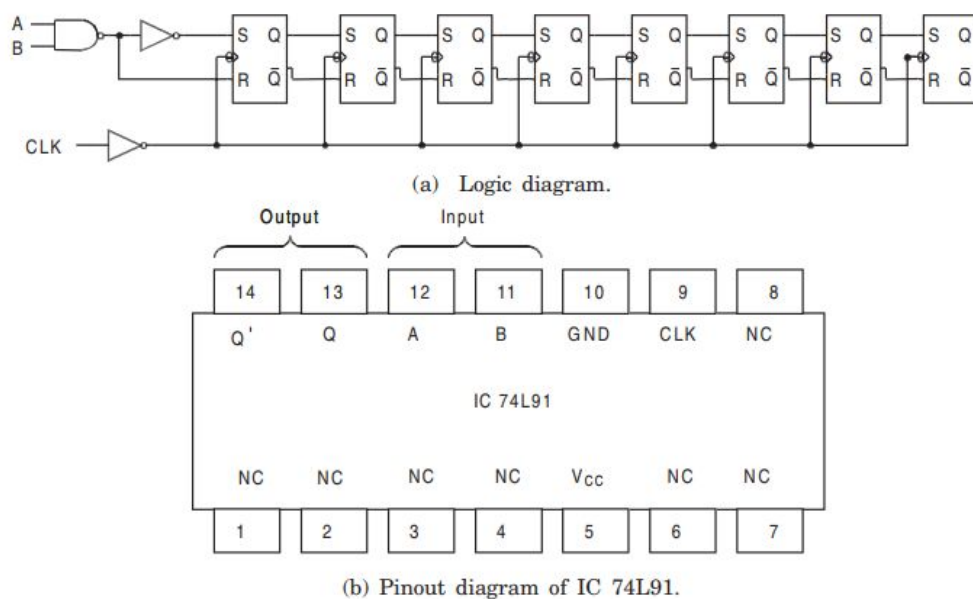


Figure 7: Eight-bit SISO register and IC 74L91 pinout from the lecture slides.

It takes four clock pulses to fill a four-bit register. If the four stored bits must then be shifted out serially, four additional edges are normally applied. The first entered bit reaches Q_D after three shifts and appears at the serial-output stage.

4 Serial-In Parallel-Out (SIPO) Register

A SIPO register has the same internal shifting connection as SISO, but every flip-flop output is available externally. The complete word can therefore be read in parallel after it has been shifted in. Thus, it operates similarly to SISO, but provides a parallel output from all register stages.

Its next-state equations are identical to SISO:

$$Q_A^+ = SI, \quad Q_B^+ = Q_A, \quad Q_C^+ = Q_B, \quad Q_D^+ = Q_C.$$

The difference is how the stored information is observed.

Feature	SISO	SIPO	Explanation
Input	Serial	Serial	One new bit enters per clock
Available out-puts	Only final stage	Every stage	SIPO exposes the complete word
Internal shifting	Same	Same	Each bit moves one position per edge

Table 5: SISO and SIPO compared.

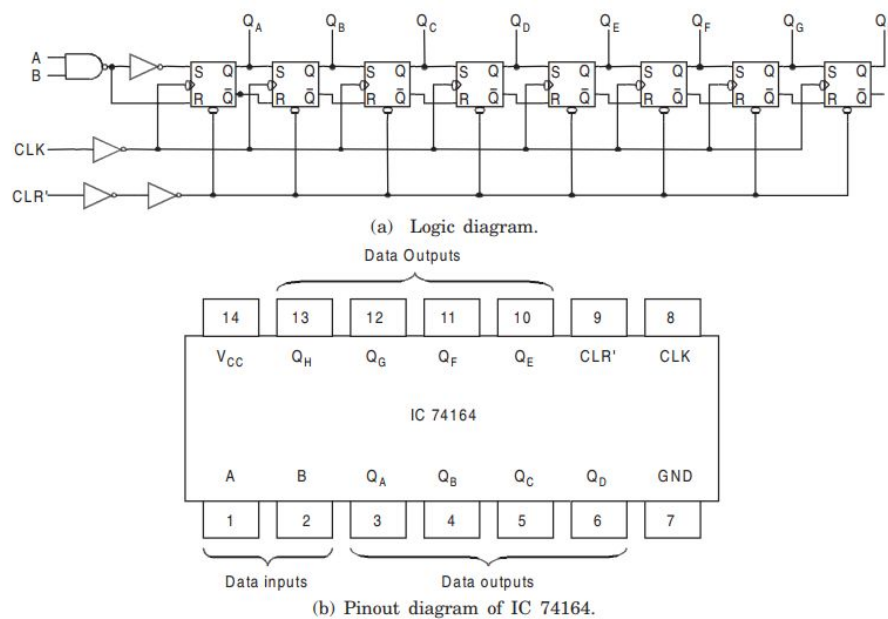


Figure 8: Eight-bit SIPO register and IC 74164 pinout from the lecture slides.

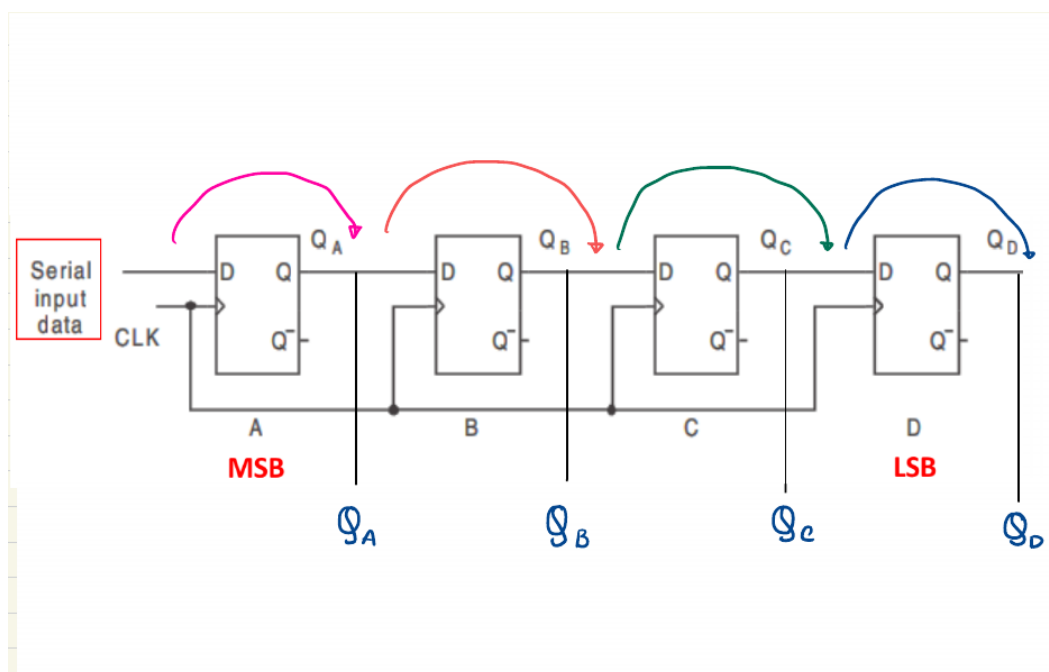


Figure 9: Annotated SIPO data path. Its shifting action is the same as SISO, but the vertical taps provide parallel outputs.

For the same input sequence 1, 1, 0, 1, the parallel outputs after the fourth edge are $Q_A Q_B Q_C Q_D = 1011$. SIPO registers are therefore useful when a serial communication stream must be presented to a parallel digital system.

5 Parallel-In Serial-Out (PISO) Register

A PISO register loads a complete word in one clock operation and then shifts the word out one bit at a time. The circuit uses a combined **Shift**/ $\overline{\text{Load}}$ control. The bar marks Load as the active-low mode: logic 0 selects Load, whereas the unbarred Shift mode is selected by logic 1. In general, whichever mode name has a bar over it is usually active when the control is low; the other mode is active when the control is high. Following the control variable S used here:

$$S = 1 \Rightarrow \text{shift}, \quad S = 0 \Rightarrow \text{parallel load}.$$

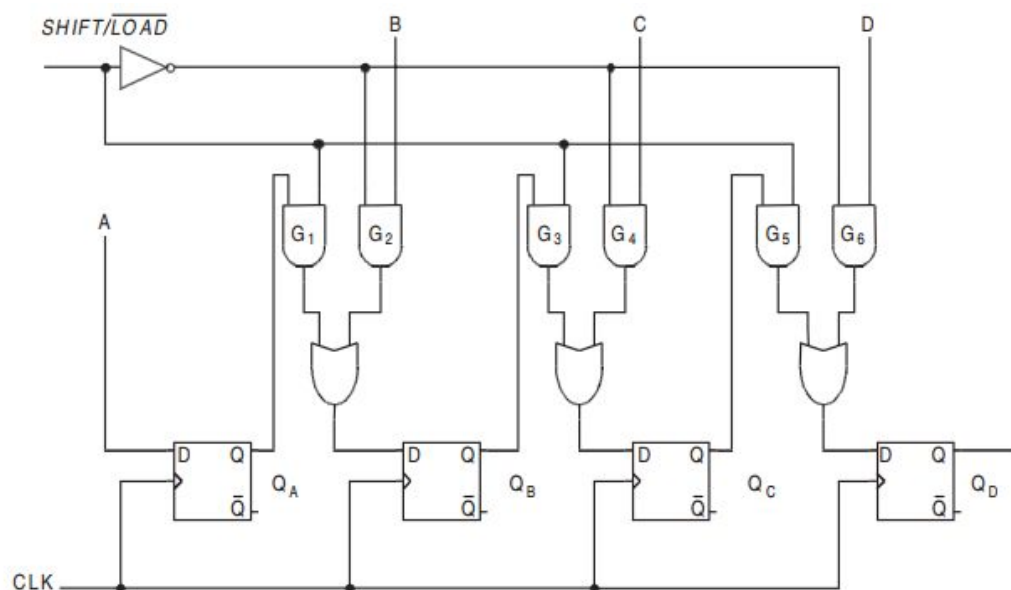


Figure 10: Clean PISO circuit from the lecture slides. It is the same selection network traced with active and inactive colors in the handwritten notes.

Each D input receives one of two possible values. When $S = 1$, the gates on the shift path are enabled. When $S = 0$, $\bar{S} = 1$, so the parallel-input path is enabled.

Color note. A thin red control line carries 0 and disables its AND gate. A blue control line carries the active value 1 and enables its AND gate; the AND-gate output then depends on the value present at its other input. The purple trace shows the selected data path.

For a right-shifting PISO register with zero entering the left end,

$$D_A = S(0) + \bar{S}A,$$

$$D_B = SQ_A + \bar{S}B,$$

$$D_C = SQ_B + \bar{S}C,$$

$$D_D = SQ_C + \bar{S}D.$$

Here A, B, C, D are the parallel inputs and Q_D is the serial output.

S	Operation	Q_A^+	Q_B^+	Q_C^+	Q_D^+
0	Parallel load	A	B	C	D
1	Shift right	0	Q_A	Q_B	Q_C

Table 6: Operation table of the illustrated PISO register.

5.1 Parallel-Load and Serial-Output Example

Let $ABCD = 1011$. First choose $S = 0$ and apply one clock edge. The register becomes 1011. Then choose $S = 1$ and apply shift clocks. The colors in Table 7 remain attached

to the four originally loaded bits, so their motion toward the serial output can be followed directly.

Edge	Operation	Q_A	Q_B	Q_C	Q_D	Serial bit available before edge
1	Load 1011	1	0	1	1	—
2	Shift	0	1	0	1	1
3	Shift	0	0	1	0	1
4	Shift	0	0	0	1	0
5	Shift	0	0	0	0	1

Table 7: Serial transmission of a parallel-loaded word.

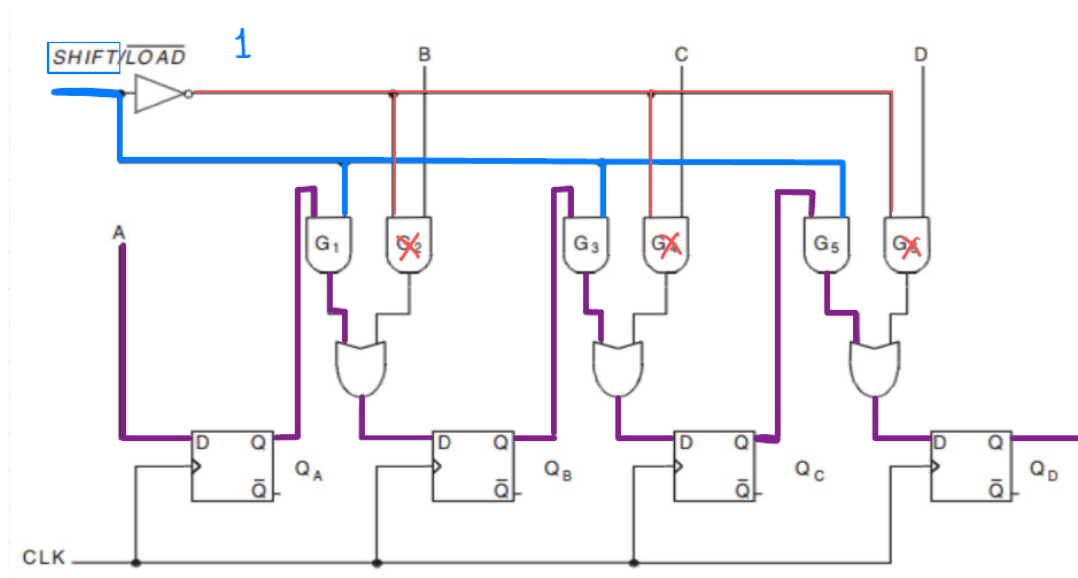


Figure 11: PISO shift mode: $S = 1$. The blue control enables the shift-path AND gates; the thin red line disables the parallel-load AND gates.

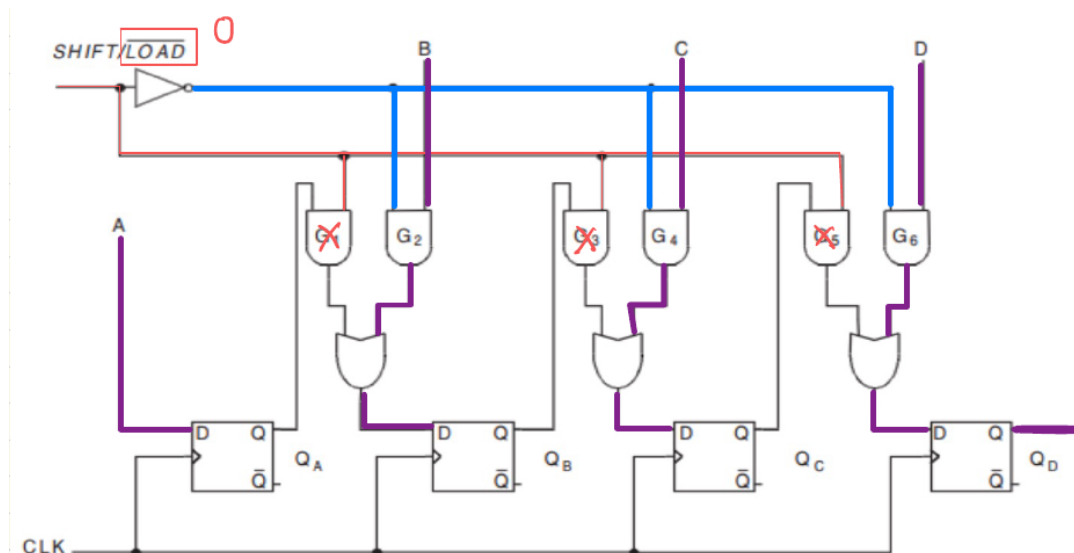
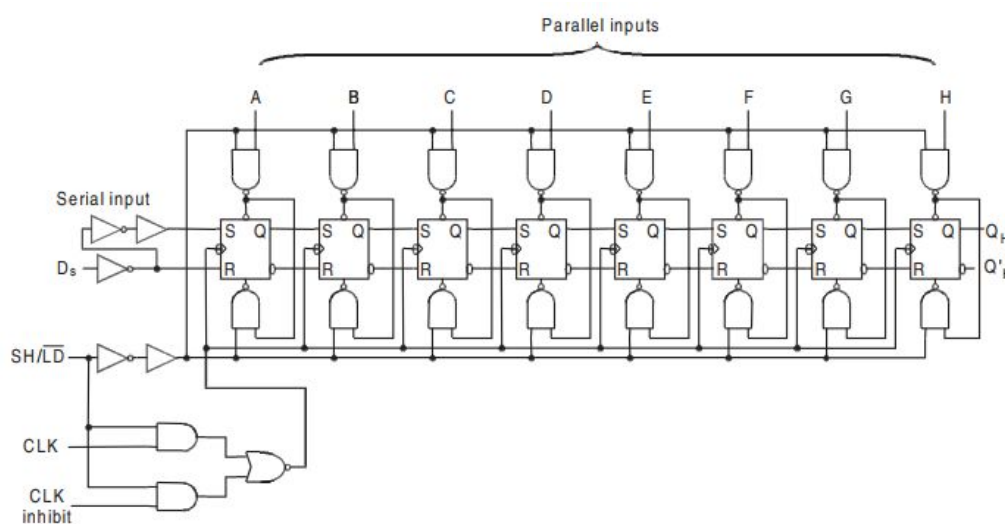
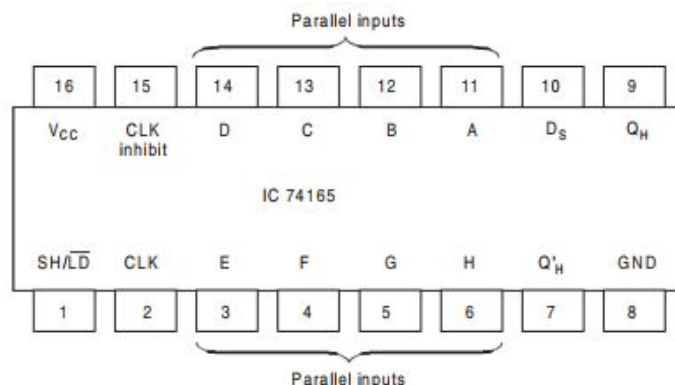


Figure 12: PISO parallel-load mode: $S = 0$ and $\bar{S} = 1$. The blue control enables the parallel-input AND gates; the thin red line disables the shift path.



(a) Logic diagram.



(b) Pinout diagram of IC 74165.

Figure 13: Eight-bit PISO implementation and IC 74165 pinout from the lecture slides.

6 Parallel-In Parallel-Out (PIPO) Register

PIPO is the simplest register structure. There is no shifting connection between adjacent stages. Each input is connected directly to its corresponding D input, and all bits are loaded at the same active clock edge.

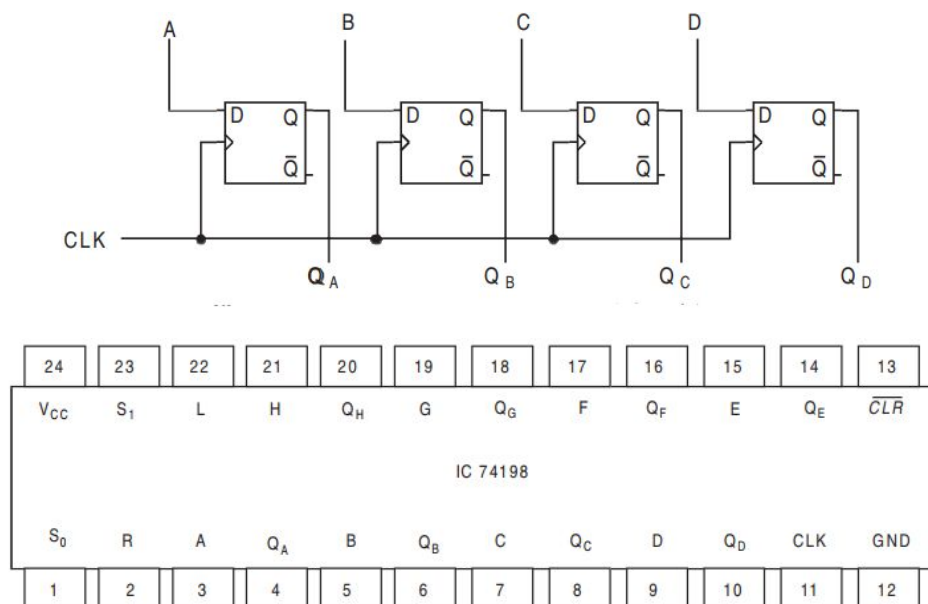


Figure 14: PIPO circuit and IC 74198 pinout from the lecture slides.

The equations are

$$Q_A^+ = A, \quad Q_B^+ = B, \quad Q_C^+ = C, \quad Q_D^+ = D.$$

Event	Before the edge	After the edge
No active clock edge	Outputs retain the old word	No change
Active clock edge	Inputs $ABCD$ are sampled	$Q_A Q_B Q_C Q_D = ABCD$
Clear asserted	Any stored word	$Q_A Q_B Q_C Q_D = 0000$

Table 8: Basic operation of a PIPO register.

PIPO registers are used as temporary storage, buffering registers, and intermediate data registers in processors and datapaths.

7 Bidirectional Shift Register

A bidirectional register can shift either right or left. The circuit from the handwritten notes places a 2-to-1 selection network before every D flip-flop. The common mode input M chooses the direction.

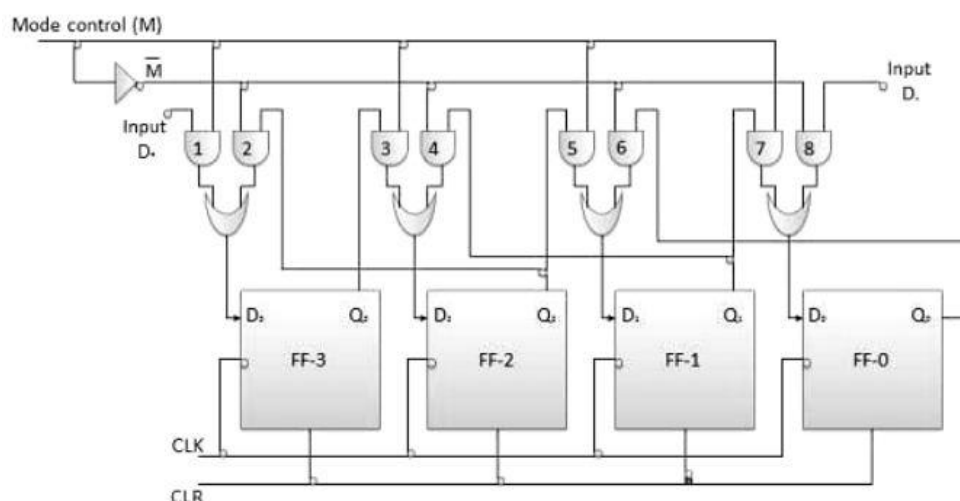


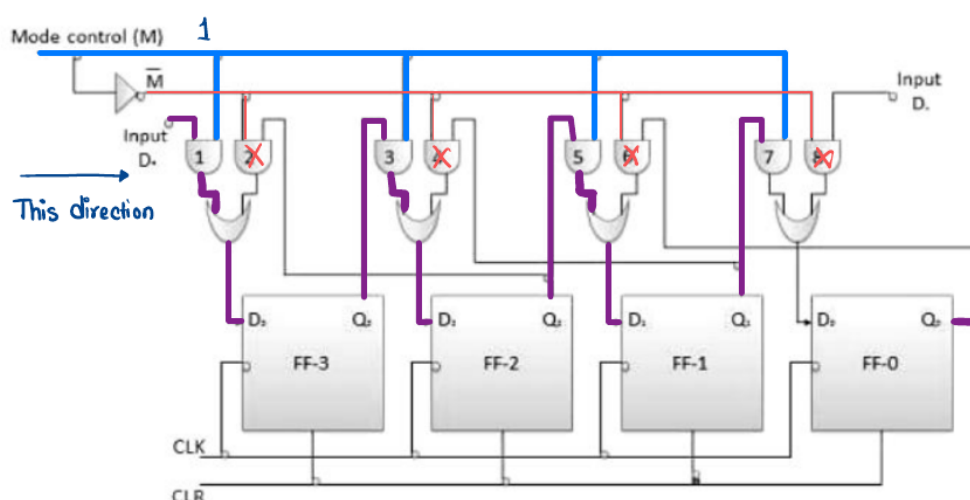
Figure 15: Clean bidirectional-register circuit from the lecture slides.

Use $Q_3Q_2Q_1Q_0$ from left to right. Let SR denote the serial input used for right shifting and SL the serial input used for left shifting. Following the handwritten mode convention:

M	Operation	Q_3^+	Q_2^+	Q_1^+	Q_0^+
1	Shift right	SR	Q_3	Q_2	Q_1
0	Shift left	Q_2	Q_1	Q_0	SL

Table 9: Mode and next-state table of the bidirectional register.

The control colors have the same meaning as in the PISO figures: blue carries the enabling 1, a thin red line carries the disabling 0, and purple traces the selected data path.

Figure 16: Bidirectional-register right-shift mode: $M = 1$. The serial input enters at the Q_3 end.

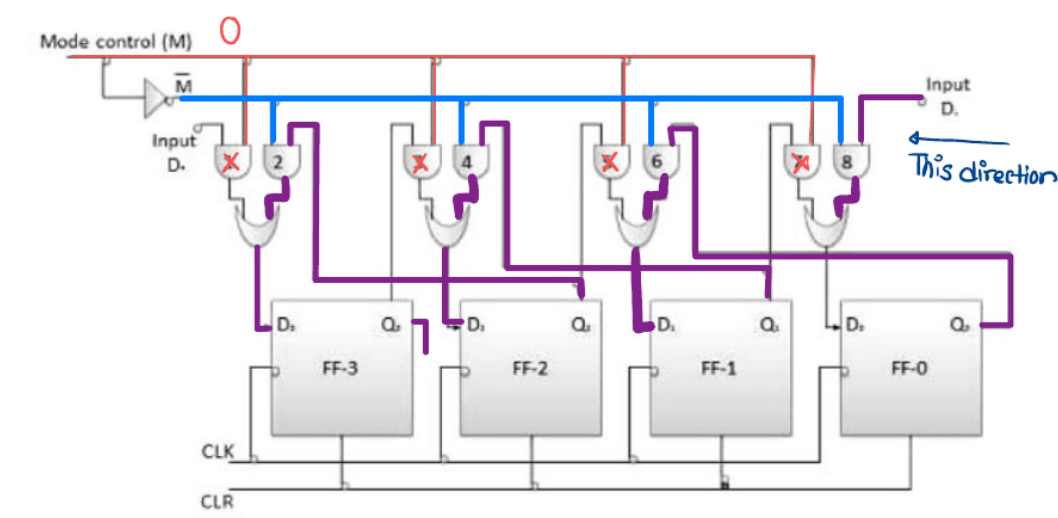


Figure 17: Bidirectional-register left-shift mode: $M = 0$ and $\overline{M} = 1$. The serial input enters at the Q_0 end.

The corresponding input equations are

$$D_3 = M(SR) + \overline{M}Q_2,$$

$$D_2 = MQ_3 + \overline{M}Q_1,$$

$$D_1 = MQ_2 + \overline{M}Q_0,$$

$$D_0 = MQ_1 + \overline{M}(SL).$$

For example, assume $Q_3Q_2Q_1Q_0 = 1011$.

- With $M = 1$ and $SR = 0$, one right shift produces 0101.
- With $M = 0$ and $SL = 0$, one left shift produces 0110.

The word does not rotate: the bit leaving one end is lost unless it is fed back externally.

8 Universal Shift Register

A universal shift register combines the important register operations in one circuit:

1. hold the present word;
2. shift right;
3. shift left;
4. load a parallel word.

Each flip-flop is preceded by a 4-to-1 multiplexer. The two common select inputs S_1S_0 choose which data source reaches every D input.

S_1	S_0	Selected operation
0	0	No change (hold)
0	1	Shift right
1	0	Shift left
1	1	Parallel load

Table 10: Universal-register mode selection.

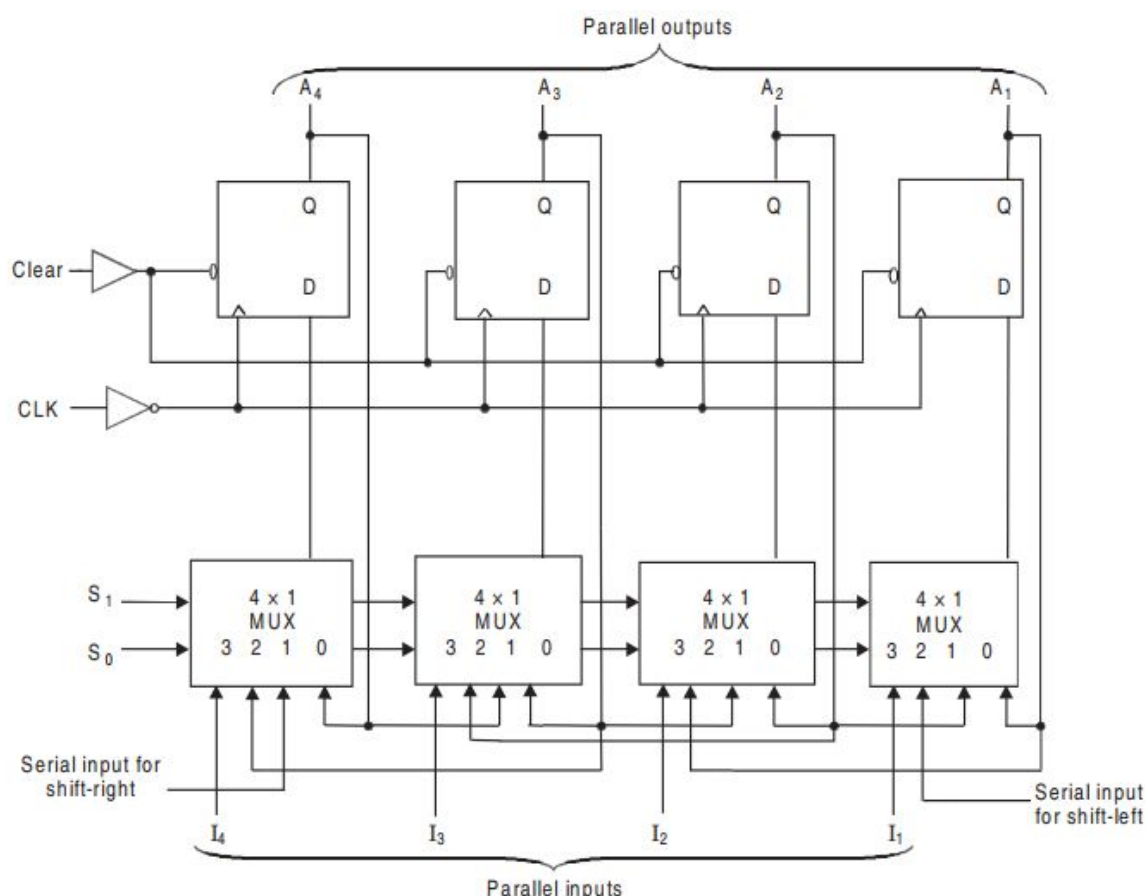


Figure 18: Four-bit universal shift register from the handwritten source notes.

In the figure, I_4, I_3, I_2, I_1 are parallel inputs, while the taps at the top are parallel outputs. Separate serial inputs supply the new boundary bit for right and left shifting. Clear resets all stages.

8.1 Mode 00: No Change

The multiplexer reconnects each flip-flop's own output to its D input:

$$Q_3^+ = Q_3, \quad Q_2^+ = Q_2, \quad Q_1^+ = Q_1, \quad Q_0^+ = Q_0.$$

This is why the pink paths in the handwritten page return vertically from each Q output to the same stage. A clock edge occurs, but the stored word remains unchanged.

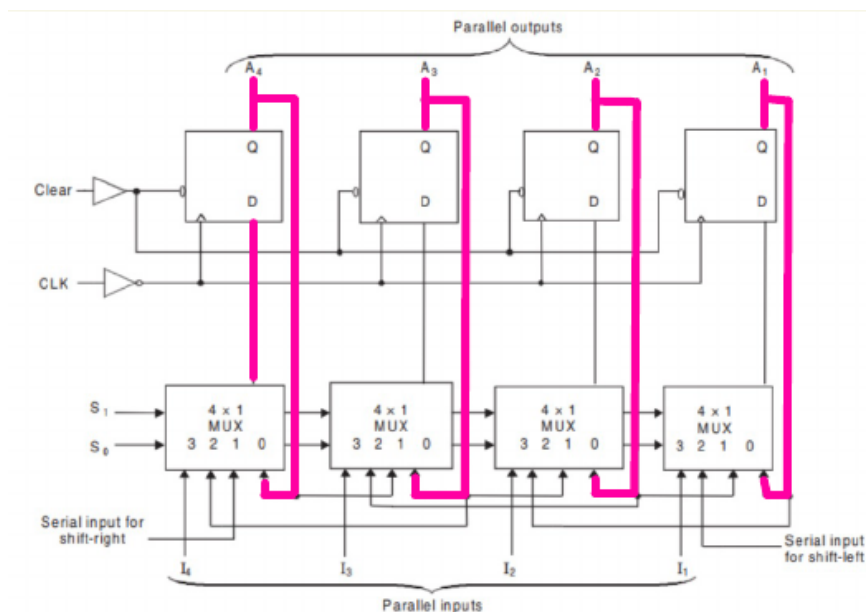


Figure 19: Universal-register hold mode ($S_1S_0 = 00$): each output is fed back to the same stage.

8.2 Mode 01: Shift Right

The green paths in the handwritten page select the right-shift connections:

$$Q_3^+ = SR, \quad Q_2^+ = Q_3, \quad Q_1^+ = Q_2, \quad Q_0^+ = Q_1.$$

The old Q_0 is shifted out, and a new bit enters at the Q_3 end.

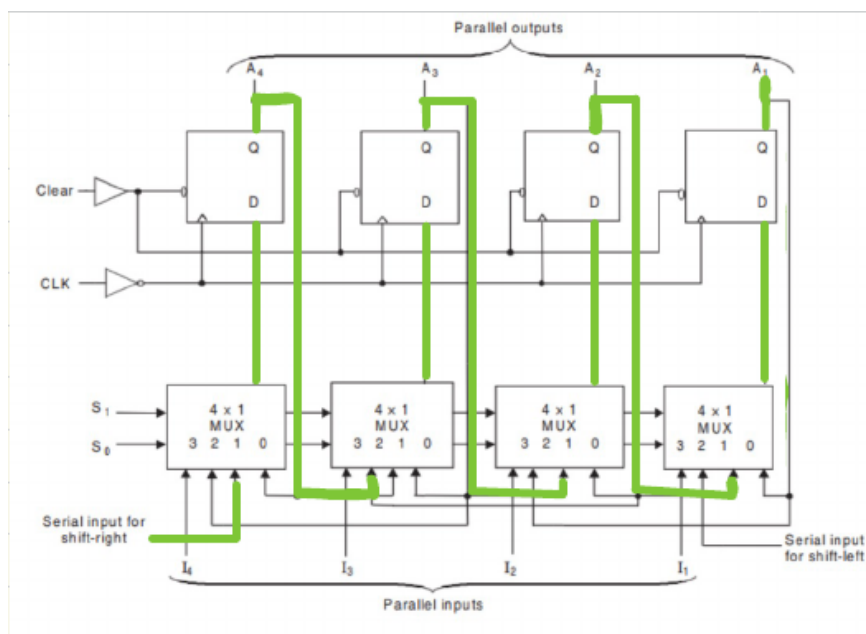


Figure 20: Universal-register right-shift mode ($S_1S_0 = 01$).

8.3 Mode 10: Shift Left

The red paths select the opposite neighbors:

$$Q_3^+ = Q_2, \quad Q_2^+ = Q_1, \quad Q_1^+ = Q_0, \quad Q_0^+ = SL.$$

The old Q_3 is shifted out, and a new bit enters at the Q_0 end.

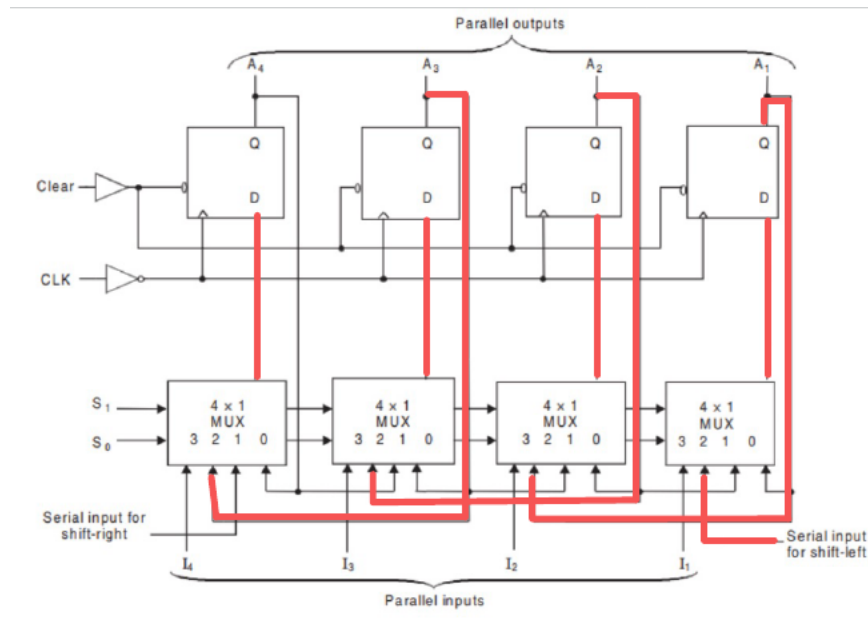


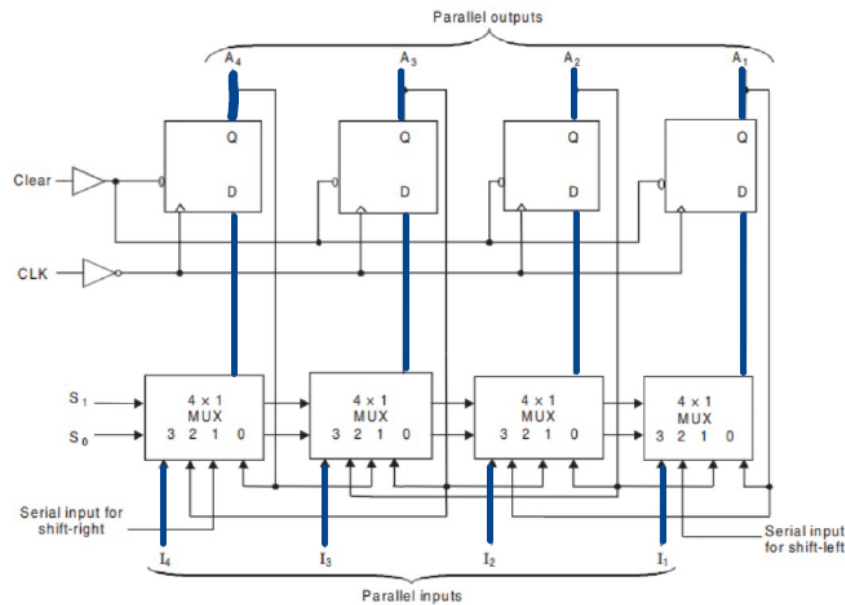
Figure 21: Universal-register left-shift mode ($S_1S_0 = 10$).

8.4 Mode 11: Parallel Load

The blue paths in the handwritten page connect the four parallel inputs to their corresponding stages:

$$Q_3^+ = I_4, \quad Q_2^+ = I_3, \quad Q_1^+ = I_2, \quad Q_0^+ = I_1.$$

All four bits are stored on the same active clock edge.

Figure 22: Universal-register parallel-load mode ($S_1S_0 = 11$).

8.5 Complete Next-State Table

S_1	S_0	Operation	D_3	D_2	D_1	D_0
0	0	Hold	Q_3	Q_2	Q_1	Q_0
0	1	Shift right	SR	Q_3	Q_2	Q_1
1	0	Shift left	Q_2	Q_1	Q_0	SL
1	1	Parallel load	I_4	I_3	I_2	I_1

Table 11: Complete input/operation table for the illustrated universal shift register.

The first-stage multiplexer equation, for example, is

$$D_3 = \overline{S_1} \overline{S_0} Q_3 + \overline{S_1} S_0 (SR) + S_1 \overline{S_0} Q_2 + S_1 S_0 I_4.$$

The other three equations follow the same pattern using the corresponding row of Table 10.

8.6 Combined Operation Example

Assume the register initially contains 0000.

Edge	S_1	S_0	Inputs and operation	New word $Q_3Q_2Q_1Q_0$
1	1	1	Parallel load $I_4I_3I_2I_1 = 1011$	1011
2	0	0	Hold	1011
3	0	1	Shift right with $SR = 0$	0101
4	1	0	Shift left with $SL = 1$	1011

Table 12: Example using all major universal-register data paths.

9 Summary Comparison

Type	Loading time	Output availability	Typical use
SISO	n clocks serially	One bit at a time	Delay line and serial transfer
SIPO	n clocks serially	All bits together after loading	Serial-to-parallel conversion
PISO	One clock in parallel	One bit per shift clock	Parallel-to-serial conversion
PIPO	One clock in parallel	All bits together	Temporary word storage
Bidirectional	One bit per shift clock	Depends on output connections	Two-way shifting
Universal	Selected by mode	Serial and parallel facilities	General-purpose data movement

Table 13: Comparison of the register structures covered in this lecture.

10 How Many Clock Cycles Must We Wait?

Let the data word contain n bits, let the register contain m stages, and let the clock period be T_{clk} . A serial connection transfers only one bit at each active clock edge, so the number of clock edges determines the waiting time.

10.1 Time to Load the Data

Condition	Loading time	What is available after loading?
$n < m$	n clocks, or nT_{clk}	All n data bits have entered, but $m - n$ stages do not belong to the word. Clear those stages first or supply padding bits when a complete m -bit value is required.
$n = m$	m clocks, or mT_{clk}	The complete word is available at the m parallel outputs.
$n > m$	n clocks to send the complete stream	One m -bit register cannot retain all n bits simultaneously. After the last edge it contains only the most recent m bits; the earlier $n - m$ bits have already shifted out. Use at least n cascaded stages, or capture the stream in $\lceil n/m \rceil$ blocks.

Table 14: Waiting time when data enters serially.

For example, an 8-bit word needs eight active edges to enter an 8-bit SIPO register. A 12-bit stream also needs twelve edges to pass through an 8-bit register, but only its most recent eight bits remain in that register.

10.2 Time to Obtain the Output

If k valid bits are already stored and must leave through one serial output, all k bits require k shift clocks. A full m -bit register therefore needs mT_{clk} to transmit its complete word serially. If serial loading and serial readout are performed as two separate phases for an n -bit word that fits in the register ($n \leq m$), the total is $2n$ clocks, excluding reset and other control operations.

With parallel loading or parallel output, all connected bits transfer together at one active edge. Thus, an m -bit parallel register handles up to m bits in one clock; data wider than m requires $\lceil n/m \rceil$ parallel blocks or multiple registers.

Part II

Counters

11 Introduction to Counters

A **counter** is a sequential circuit that changes state in a prescribed sequence whenever clock pulses arrive. Counters are used to count events, divide frequency, measure time, and generate control sequences. An m -bit binary counter can represent at most 2^m distinct states.

11.1 Why Counters Usually Use T Flip-Flops

Counting repeatedly requires stored bits to change between 0 and 1. A T flip-flop performs exactly this operation:

$$T = 0 \Rightarrow Q^+ = Q, \quad T = 1 \Rightarrow Q^+ = \overline{Q}.$$

Therefore, tying $T = 1$ makes a stage toggle whenever its active clock edge arrives. A JK flip-flop behaves in the same way when $J = K = 1$. D and SR flip-flops can also implement counters, but normally require extra logic to generate their inputs.

11.2 Types of Counters

Classification	Main idea
Asynchronous (ripple)	Only the first flip-flop receives the external clock; every later stage is clocked by the preceding stage.
Synchronous	All flip-flops receive the common clock; input logic decides which stages toggle.
Up / down	The state sequence increases or decreases by one.
Single-mode / multi-mode	A single-mode counter has one direction; a multimode counter can select between operations such as up and down.
MOD- N	The sequence contains N distinct states before repeating.
Ring / Johnson	Shift-register counters that circulate direct or complemented feedback.

Table 15: Common ways to classify counters.

Detailed synchronous binary and up/down counters were discussed in class and are not repeated here. The synchronous sequential-circuit design procedure is also assumed to be known.

12 Three-Bit Asynchronous Up Counter

Figure 23 uses three negative-edge-triggered T flip-flops with $T = 1$. The external clock drives only Q_0 , the LSB stage. Then Q_0 clocks Q_1 , and Q_1 clocks Q_2 . Consequently, the clock seen by each stage is different:

$$\text{CLK}_0 = \text{CLK}, \quad \text{CLK}_1 = Q_0, \quad \text{CLK}_2 = Q_1.$$

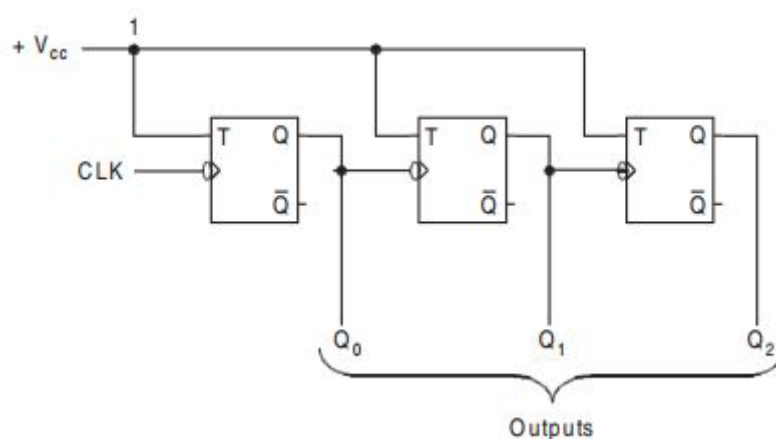


Figure 23: Three-bit asynchronous up counter using negative-edge-triggered T flip-flops. Each Q output supplies the clock of the following stage.

12.1 How the Negative-Edge Ripple Works

At every falling edge of the external clock, Q_0 toggles. Flip-flop 1 toggles only when Q_0 makes a $1 \rightarrow 0$ transition, and flip-flop 2 toggles only when Q_1 makes a $1 \rightarrow 0$ transition. In Table 16, red tracks Q_0 , blue tracks Q_1 , and green tracks Q_2 .

External falling edge	Clock of FF0	Clock of FF1	Clock of FF2	Changes produced in ripple order	Settled $Q_2Q_1Q_0$
0	—	—	—	Initial state	000
1	↓CLK	—	—	$Q_0 : 0 \rightarrow 1$	001
2	↓CLK	↓ Q_0	—	$Q_0 : 1 \rightarrow 0$, then $Q_1 : 0 \rightarrow 1$	010
3	↓CLK	—	—	$Q_0 : 0 \rightarrow 1$	011
4	↓CLK	↓ Q_0	↓ Q_1	$Q_0 : 1 \rightarrow 0$, then $Q_1 : 1 \rightarrow 0$, then $Q_2 : 0 \rightarrow 1$	100
5	↓CLK	—	—	$Q_0 : 0 \rightarrow 1$	101
6	↓CLK	↓ Q_0	—	$Q_0 : 1 \rightarrow 0$, then $Q_1 : 0 \rightarrow 1$	110
7	↓CLK	—	—	$Q_0 : 0 \rightarrow 1$	111
8	↓CLK	↓ Q_0	↓ Q_1	$Q_0 : 1 \rightarrow 0$, then $Q_1 : 1 \rightarrow 0$, then $Q_2 : 1 \rightarrow 0$	000

Table 16: The clock received by each flip-flop and the resulting state change.

The word *asynchronous* is important: the three changes at edges 4 and 8 do not occur at exactly the same instant. Each change waits for the propagation delay of the previous flip-flop. Thus, at edge 4 the circuit briefly ripples through intermediate values before settling at 100.

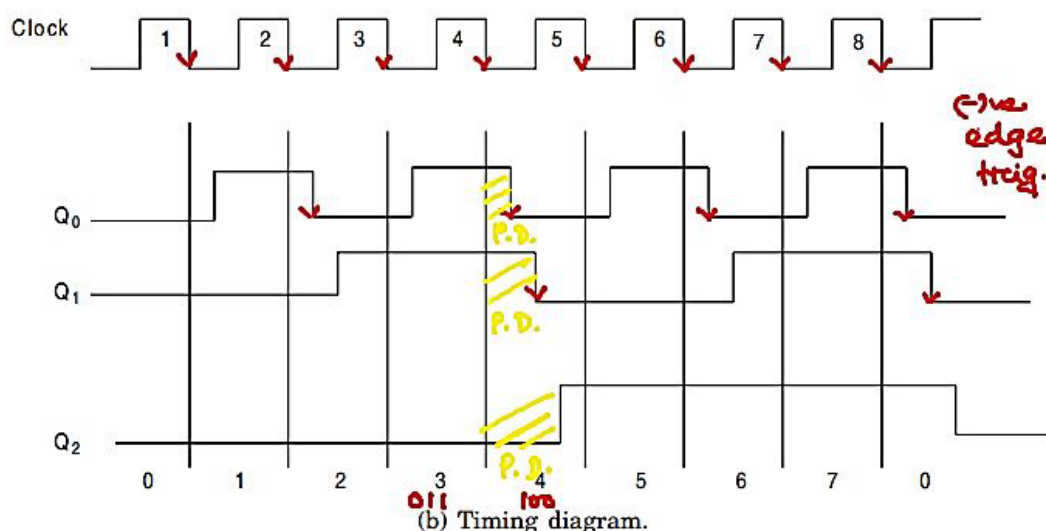


Figure 24: Timing diagram of the negative-edge-triggered ripple up counter. The red arrows mark active falling edges; the highlighted region shows the accumulated propagation delay near the transition from 011 to 100.

Each stage divides its input-clock frequency by two:

$$f_{Q_0} = \frac{f_{\text{CLK}}}{2}, \quad f_{Q_1} = \frac{f_{\text{CLK}}}{4}, \quad f_{Q_2} = \frac{f_{\text{CLK}}}{8}.$$

For an m -stage ripple counter with flip-flop propagation delay t_{pd} , the worst-case settling time is approximately mt_{pd} .

13 Three-Bit Asynchronous Down Counter

The descending sequence is

$$111 \rightarrow 110 \rightarrow 101 \rightarrow 100 \rightarrow 011 \rightarrow 010 \rightarrow 001 \rightarrow 000 \rightarrow 111.$$

Falling edge	0	1	2	3	4	5	6	7	8
$Q_2Q_1Q_0$	111	110	101	100	011	010	001	000	111
Decimal	7	6	5	4	3	2	1	0	7

Table 17: Count sequence of a three-bit asynchronous down counter.

Only the up-counter circuit is drawn in detail here. A ripple down counter can be obtained in any of the following common ways:

1. keep the up-counter connections but read the complemented outputs $\overline{Q_2}\overline{Q_1}\overline{Q_0}$;
2. with negative-edge-triggered flip-flops, clock each later stage from the preceding $\overline{Q}$ output instead of Q ;
3. keep the Q -to-clock connections but use positive-edge-triggered flip-flops.

13.1 Way 2: Clock the Next Stage from $\overline{Q}$

Figure 25 shows the second method. All T inputs remain at 1 and the flip-flops remain negative-edge triggered, but $\overline{Q}_0$ clocks the Q_1 stage and $\overline{Q}_1$ clocks the Q_2 stage. Starting from 000, the first falling edge makes Q_0 rise; therefore $\overline{Q}_0$ falls and triggers Q_1 . The same ripple reaches Q_2 , producing 111. Later pulses continue the descending sequence.

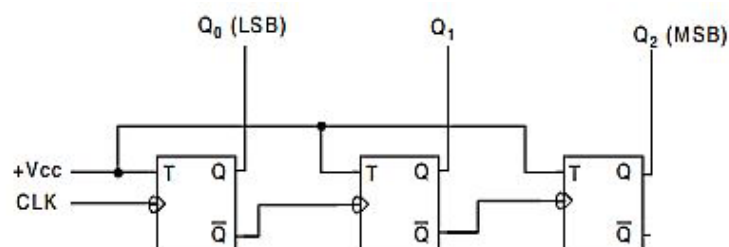


Figure 25: Three-bit asynchronous down counter, Way 2: each negative-edge-triggered stage is clocked by the preceding complemented output.

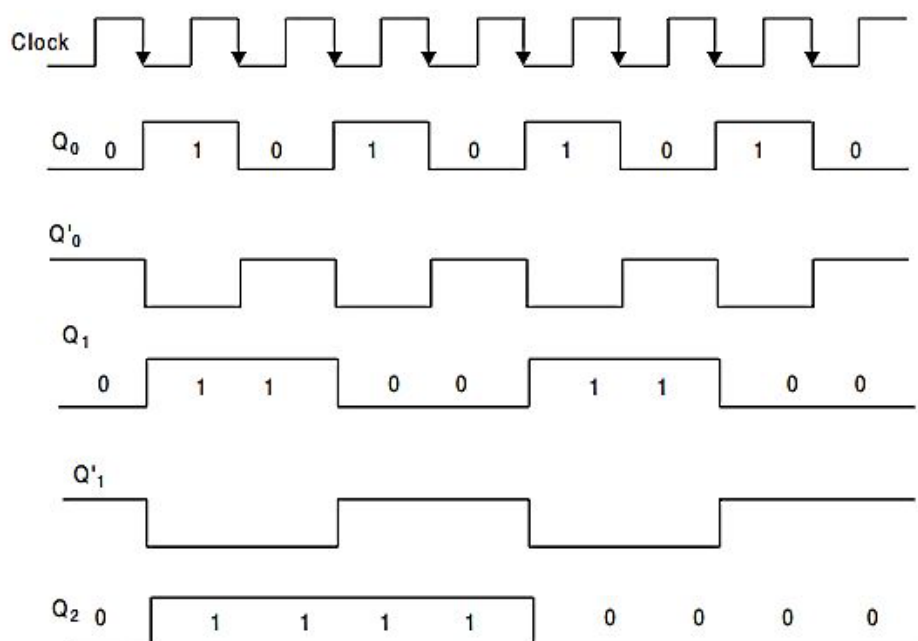


Figure 26: Timing diagram for Way 2. The falling edges of $\overline{Q}_0$ and $\overline{Q}_1$ generate the ripple that produces the down-count sequence.

Always state both the clock edge and whether Q or $\overline{Q}$ drives the next stage; these choices determine the counting direction.

14 MOD Counters

The modulus is the number of stable states in the repeating sequence. A full m -bit binary counter is MOD- 2^m . A MOD- N counter requires at least

$$m = \lceil \log_2 N \rceil$$

flip-flops. When $N < 2^m$, decoding logic recognizes the first unwanted state and returns the counter to its initial state.

14.1 MOD-5 Counter and Its Reset Logic

The provided Proteus circuit uses 74107 JK flip-flops with $J = K = 1$, so each stage toggles like a T flip-flop. Its 4023 device is a three-input **NAND** gate, even though its body resembles an AND gate. The gate combines selected counter outputs to recognize the reset state. Its low output is appropriate for the active-low clear inputs of the 74107 flip-flops.

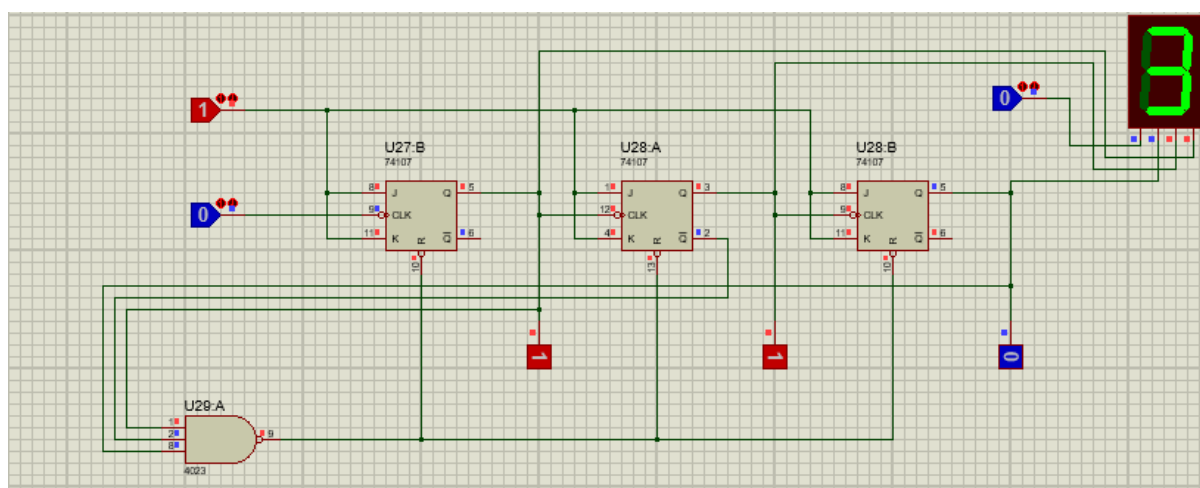


Figure 27: Provided Proteus implementation of the three-bit MOD-5 counter. The 4023 NAND reset network decodes $Q_2Q_1Q_0 = 101$.

The stable count sequence is

$$000 \rightarrow 001 \rightarrow 010 \rightarrow 011 \rightarrow 100 \rightarrow 000.$$

Thus, the display counts 0, 1, 2, 3, 4 and repeats. On the next pulse, the ripple counter momentarily attempts to form $101_2 = 5$. At that instant $Q_2 = Q_0 = 1$, and $Q_1 = 0$, so

$$\overline{\text{CLR}} = \overline{Q_2Q_1Q_0} = \overline{(1)(0)(1)} = 0.$$

The low signal activates the asynchronous clear inputs and immediately forces $Q_2Q_1Q_0 = 000$. The 101 state is therefore a short reset condition, not a stable displayed count. Including $\overline{Q_1}$ makes this an exact decoder: 111 does not activate the reset because $\overline{Q_1} = 0$ there. An AND decoder followed by an inverter is logically equivalent when the clear input is active-low.

Decimal	Q_2	Q_1	Q_0	$Q_2\overline{Q_1}Q_0$	$\overline{\text{CLR}}$
0	0	0	0	0	1
1	0	0	1	0	1
2	0	1	0	0	1
3	0	1	1	0	1
4	1	0	0	0	1
5 (transient)	1	0	1	1	0 $\rightarrow$ clear to 000
6 (unused)	1	1	0	0	1
7 (unused)	1	1	1	0	1

Table 18: Why the NAND reset decoder makes the counter MOD-5.

Because this reset is asynchronous, the decoded state can appear briefly and the outputs can ripple before clearing. Do not treat those transition values as stable counts.

14.2 Synchronous MOD Counters

A synchronous MOD counter is designed using the usual synchronous sequential circuit procedure: define the N -state sequence, construct the present/next state and excitation tables, simplify the flip-flop input equations, and decide how unused states recover. Since that procedure was already covered in class, it is not repeated here.

15 Ring and Johnson Counters

15.1 Ring Counter

A ring counter is a shift register whose final Q output is fed directly back to the first D input. It must be initialized with a one-hot word such as 1000. The single 1 then circulates through the register.

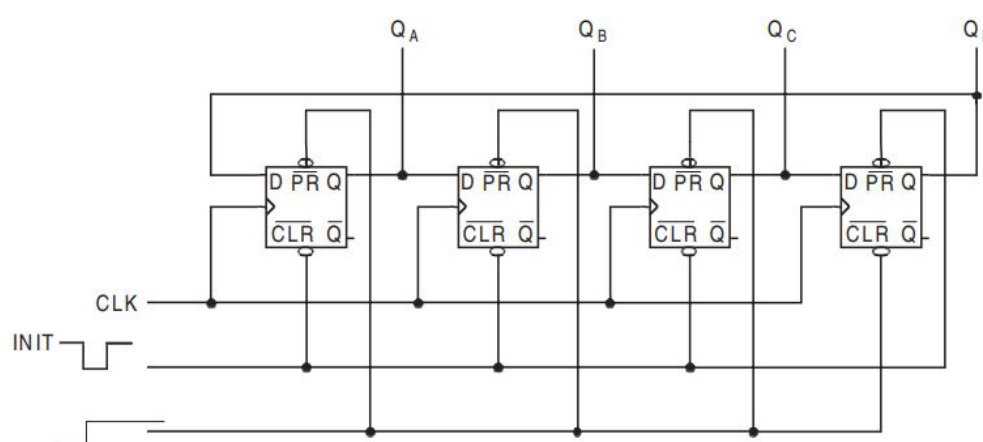


Figure 28: Four-bit ring counter using D flip-flops. The initialization network places one stage at 1 and the remaining stages at 0. The source figure labels its final output Q_E ; it is the fourth stage, denoted Q_D in the table.

Clock	Q_A	Q_B	Q_C	Q_D	State
0	1	0	0	0	1000
1	0	1	0	0	0100
2	0	0	1	0	0010
3	0	0	0	1	0001
4	1	0	0	0	1000

Table 19: The colored 1 circulating in a four-bit ring counter.

An m -bit ring counter normally has m valid one-hot states, so it acts as a MOD- m sequence generator. Incorrect initialization such as 0000 remains locked unless extra recovery logic is provided.

15.2 Johnson Counter

A Johnson counter, or twisted-ring counter, feeds the complemented output of the last stage back to the first stage. Zeros first enter as a growing run of ones; after all stages become 1, zeros enter again.

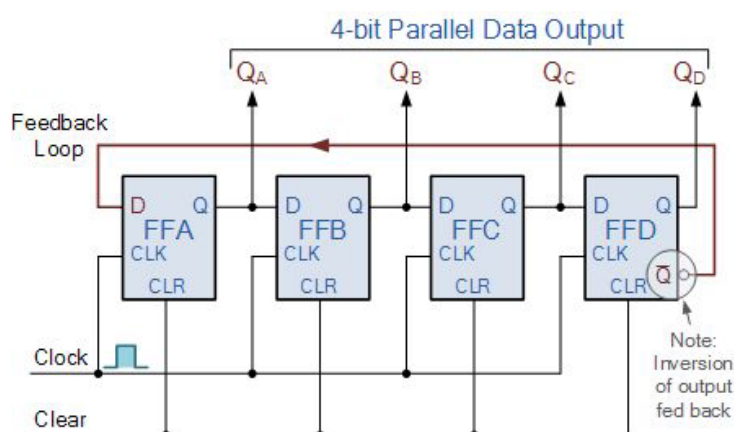


Figure 29: Four-bit Johnson counter. The complemented last-stage output is fed back to the first D input.

Clock	0	1	2	3	4	5	6	7	8
State	0000	1000	1100	1110	1111	0111	0011	0001	0000

Table 20: Eight-state sequence of a four-bit Johnson counter. Red shows ones entering; blue shows zeros returning.

An m -bit Johnson counter provides $2m$ valid states, twice as many as a ring counter with the same number of flip-flops. Ring and Johnson counters are useful for simple timing and sequence generation, but neither replaces a general binary counter.

16 Counter Topics Already Covered in Class

The detailed circuits for asynchronous up/down counters, synchronous binary counters, synchronous up/down counters, and the full synchronous sequential design procedure are intentionally omitted. For this document, remember only that synchronous counters clock every flip-flop together, while combinational input logic determines which stages toggle at that edge.

17 A few things to note

- Understand the difference between Bidirectional and Universal shift registers. They have different purposes.
- The mode values of Bidirectional and Universal shift registers are different. For example, for bidirectional shift register, $M = 1$ doesn't always mean it will shift right, it depends on how you are designing it, the circuit might shift right if $M = 0$ if you design it that way. Unless anything is mentioned, you are free to design it in any way you want.
- Try simulating the registers in Proteus to understand how they work. You will find them here - [Proteus Files](#)